# ALLIANCE UNIVERSITY

## Database Management System

### Assignment 9

---

# CAB BOOKING SYSTEM



| | |
|---|---|
| **Name** | Razin Basha R |
| **Roll No** | 2511021240012 |
| **Course** | Database Management System |
| **Programme** | BCA AI/ML |
| **Semester** | III |

## Reports using joins

### Report 1: Complete booking details (INNER JOIN on four tables)

```
SELECT b.Booking_ID, c.Customer_Name, car.Car_Model, d.Driver_Name,
       b.Pickup_Location, b.Drop_Location, b.Distance_KM, b.Booking_Status
FROM Booking b
INNER JOIN Customer c ON b.Customer_ID = c.Customer_ID
INNER JOIN Car car    ON b.Car_ID      = car.Car_ID
INNER JOIN Driver d   ON car.Driver_ID = d.Driver_ID
ORDER BY b.Booking_ID;
```

**Output:**

| Booking_ID | Customer_Name | Car_Model | Driver_Name | Pickup_Location | Drop_Location | Distance_KM | Booking_Status |
|---|---|---|---|---|---|---|---|
| B01 | Rahul | Swift | Ravi | Airport | MG Road | 25 | Completed |
| B02 | Arjun | City | Kumar | Mall | Railway Station | 12 | Completed |
| B03 | Kiran | Creta | Suresh | College | Airport | 30 | Confirmed |
| B04 | Priya | Innova | Mohan | Whitefield | Airport | 38 | Completed |
| B05 | Rahul | Dzire | Prakash | MG Road | Electronic City | 22 | Completed |
| B06 | Meena | Alto | Selvam | Railway Station | T Nagar | 8 | Completed |
| B07 | Arjun | Creta | Suresh | Airport | Hosur | 45 | Completed |
| B08 | Vikram | City | Kumar | Koramangala | Indiranagar | 9 | Cancelled |
| B09 | Priya | Swift | Ravi | Airport | Whitefield | 36 | Completed |
| B10 | Divya | Baleno | Ravi | Mall | College | 6 | Completed |
| B11 | Rahul | Innova | Mohan | Electronic City | Airport | 40 | Completed |
| B12 | Kiran | Dzire | Prakash | Krishnagiri | Hosur | 35 | Completed |
| B13 | Meena | City | Kumar | T Nagar | Airport | 21 | Cancelled |
| B14 | Vikram | Creta | Suresh | Indiranagar | Airport | 33 | Confirmed |
| B15 | Arjun | Alto | Selvam | MG Road | Mall | 5 | Completed |
| B16 | Divya | Innova | Mohan | College | Whitefield | 28 | Confirmed |

### Report 2: Pending payments with customer details

```
SELECT p.Payment_ID, b.Booking_ID, c.Customer_Name, c.Phone,
       p.Amount, p.Payment_Method
FROM Payment p
JOIN Booking b  ON p.Booking_ID  = b.Booking_ID
JOIN Customer c ON b.Customer_ID = c.Customer_ID
WHERE p.Payment_Status = 'Pending'
ORDER BY p.Amount DESC;
```

**Output:**

| Payment_ID | Booking_ID | Customer_Name | Phone | Amount | Payment_Method |
|---|---|---|---|---|---|
| P14 | B14 | Vikram | 9876543215 | 660.00 | UPI |
| P16 | B16 | Divya | 9876543216 | 616.00 | Card |
| P03 | B03 | Kiran | 9876543212 | 600.00 | UPI |

### Report 3: Every customer with number of bookings and money spent (LEFT JOIN)

```
SELECT c.Customer_ID, c.Customer_Name,
       COUNT(b.Booking_ID)        AS Total_Bookings,
       COALESCE(SUM(p.Amount), 0) AS Total_Spent
FROM Customer c
LEFT JOIN Booking b ON c.Customer_ID = b.Customer_ID
LEFT JOIN Payment p ON p.Booking_ID  = b.Booking_ID
GROUP BY c.Customer_ID, c.Customer_Name
ORDER BY Total_Spent DESC, c.Customer_ID;
```

**Output:**

| Customer_ID | Customer_Name | Total_Bookings | Total_Spent |
|---|---|---|---|
| CU01 | Rahul | 3 | 1488.00 |
| CU04 | Priya | 2 | 1268.00 |
| CU02 | Arjun | 3 | 1130.00 |
| CU03 | Kiran | 2 | 1090.00 |
| CU07 | Divya | 2 | 682.00 |
| CU06 | Vikram | 2 | 660.00 |
| CU05 | Meena | 2 | 80.00 |
| CU08 | Manoj | 0 | 0.00 |

## Report 4: Earnings of each driver from paid trips

```
SELECT d.Driver_ID, d.Driver_Name,
       COUNT(p.Payment_ID)  AS Paid_Trips,
       SUM(p.Amount)        AS Total_Earnings,
       ROUND(AVG(p.Amount),2) AS Avg_Per_Trip
FROM Driver d
JOIN Car car    ON car.Driver_ID = d.Driver_ID
JOIN Booking b  ON b.Car_ID      = car.Car_ID
JOIN Payment p  ON p.Booking ID  = b.Booking ID
WHERE p.Payment_Status = 'Paid'
GROUP BY d.Driver_ID, d.Driver_Name
ORDER BY Total_Earnings DESC;
```

**Output:**

| Driver_ID | Driver_Name | Paid_Trips | Total_Earnings | Avg_Per_Trip |
|---|---|---|---|---|
| D04 | Mohan | 2 | 1716.00 | 858.00 |
| D03 | Suresh | 1 | 900.00 | 900.00 |
| D01 | Ravi | 3 | 798.00 | 266.00 |
| D05 | Prakash | 2 | 798.00 | 399.00 |
| D02 | Kumar | 1 | 180.00 | 180.00 |
| D06 | Selvam | 2 | 130.00 | 65.00 |

## Report 5: Revenue and average distance for each car type

```
SELECT car.Car_Type,
       COUNT(b.Booking_ID)       AS Trips,
       SUM(p.Amount)             AS Revenue,
       ROUND(AVG(b.Distance_KM),2) AS Avg_Distance_KM
FROM Car car
JOIN Booking b ON b.Car_ID     = car.Car_ID
JOIN Payment p ON p.Booking_ID = b.Booking_ID
WHERE p.Payment_Status = 'Paid'
GROUP BY car.Car_Type
ORDER BY Revenue DESC;
```

**Output:**

| Car_Type | Trips | Revenue | Avg_Distance_KM |
|---|---|---|---|
| SUV | 3 | 2616.00 | 41.00 |
| Sedan | 3 | 978.00 | 23.00 |
| Hatchback | 5 | 928.00 | 16.00 |

## Report 6: Cars that have never been booked or are unused (LEFT JOIN with IS NULL)

```
SELECT car.Car_ID, car.Car_Model, car.Car_Type
FROM Car car
LEFT JOIN Booking b ON b.Car_ID = car.Car_ID
WHERE b.Booking_ID IS NULL;
```

**Output:**

```
Empty set (0 rows)
```

## Reports using subqueries

### Report 7: Bookings longer than the average trip (scalar subquery)

```
SELECT Booking_ID, Customer_ID, Pickup_Location, Drop_Location, Distance_KM
FROM Booking
WHERE Distance_KM > (SELECT AVG(Distance_KM) FROM Booking)
ORDER BY Distance_KM DESC;
```

**Output:**

| Booking_ID | Customer_ID | Pickup_Location | Drop_Location | Distance_KM |
|---|---|---|---|---|
| B07 | CU02 | Airport | Hosur | 45 |
| B11 | CU01 | Electronic City | Airport | 40 |
| B04 | CU04 | Whitefield | Airport | 38 |
| B09 | CU04 | Airport | Whitefield | 36 |
| B12 | CU03 | Krishnagiri | Hosur | 35 |
| B14 | CU06 | Indiranagar | Airport | 33 |
| B03 | CU03 | College | Airport | 30 |
| B16 | CU07 | College | Whitefield | 28 |
| B01 | CU01 | Airport | MG Road | 25 |

### Report 8: Customers who have travelled in an SUV (IN subquery)

```
SELECT Customer_ID, Customer_Name, Address
FROM Customer
WHERE Customer_ID IN (
    SELECT b.Customer_ID
    FROM Booking b
    JOIN Car car ON b.Car_ID = car.Car_ID
    WHERE car.Car_Type = 'SUV'
)
ORDER BY Customer_ID;
```

**Output:**

| Customer_ID | Customer_Name | Address |
|---|---|---|
| CU01 | Rahul | Bangalore |
| CU02 | Arjun | Chennai |
| CU03 | Kiran | Krishnagiri |
| CU04 | Priya | Bangalore |
| CU06 | Vikram | Bangalore |
| CU07 | Divya | Hosur |

### Report 9: Customers who have never made a booking (NOT EXISTS)

```
SELECT Customer_ID, Customer_Name, Phone
FROM Customer c
WHERE NOT EXISTS (
    SELECT 1 FROM Booking b WHERE b.Customer_ID = c.Customer_ID
);
```

**Output:**

| Customer_ID | Customer_Name | Phone |
|---|---|---|
| CU08 | Manoj | 9876543217 |

## Report 10: Longest trip of every customer (subquery in SELECT)

```
SELECT c.Customer_Name,
       (SELECT MAX(b.Distance_KM)
        FROM Booking b
        WHERE b.Customer_ID = c.Customer_ID) AS Longest_Trip_KM
FROM Customer c
ORDER BY Longest_Trip_KM DESC, c.Customer_Name;
```

**Output:**

| Customer_Name | Longest_Trip_KM |
|---|---|
| Arjun | 45 |
| Rahul | 40 |
| Priya | 38 |
| Kiran | 35 |
| Vikram | 33 |
| Divya | 28 |
| Meena | 21 |
| Manoj | NULL |

## Report 11: Customers who paid more than the average payment (subquery in HAVING)

```
SELECT c.Customer ID, c.Customer Name, SUM(p.Amount) AS Total Paid
FROM Customer c
JOIN Booking b ON b.Customer_ID = c.Customer_ID
JOIN Payment p ON p.Booking_ID  = b.Booking_ID
WHERE p.Payment_Status = 'Paid'
GROUP BY c.Customer_ID, c.Customer_Name
HAVING SUM(p.Amount) > (SELECT AVG(Amount)
                        FROM Payment
                        WHERE Payment_Status = 'Paid')
ORDER BY Total_Paid DESC;
```

**Output:**

| Customer_ID | Customer_Name | Total_Paid |
|---|---|---|
| CU01 | Rahul | 1488.00 |
| CU04 | Priya | 1268.00 |
| CU02 | Arjun | 1130.00 |
| CU03 | Kiran | 490.00 |

## Report 12: Driver with the highest earnings (subquery with a derived table)

```
SELECT t.Driver_Name, t.Total_Earnings
FROM (
    SELECT d.Driver_Name, SUM(p.Amount) AS Total_Earnings
    FROM Driver d
    JOIN Car car   ON car.Driver_ID = d.Driver_ID
    JOIN Booking b ON b.Car_ID      = car.Car_ID
    JOIN Payment p ON p.Booking_ID  = b.Booking_ID
    WHERE p.Payment_Status = 'Paid'
    GROUP BY d.Driver_Name
) AS t
WHERE t.Total_Earnings = (
    SELECT MAX(x.Total_Earnings)
    FROM (
```

```
        SELECT SUM(p.Amount) AS Total_Earnings
        FROM Driver d
        JOIN Car car   ON car.Driver_ID = d.Driver_ID
        JOIN Booking b ON b.Car_ID      = car.Car_ID
        JOIN Payment p ON p.Booking_ID  = b.Booking_ID
        WHERE p.Payment_Status = 'Paid'
        GROUP BY d.Driver_Name
    ) AS x
);
```

**Output:**

| Driver_Name | Total_Earnings |
|---|---|
| Mohan | 1716.00 |

## Summary report for the whole system

### Report 13: Management summary

```
SELECT
  (SELECT COUNT(*) FROM Customer)                                  AS Customers,
  (SELECT COUNT(*) FROM Driver WHERE Availability = 'Available') AS Free_Drivers,
  (SELECT COUNT(*) FROM Booking)                                   AS Bookings,
  (SELECT COUNT(*) FROM Booking
   WHERE Booking_Status = 'Cancelled')                             AS Cancelled,
  (SELECT SUM(Amount) FROM Payment
   WHERE Payment_Status = 'Paid')                                  AS Revenue_Received,
  (SELECT SUM(Amount) FROM Payment
   WHERE Payment_Status = 'Pending')                               AS Revenue_Pending;
```

**Output:**

| Customers | Free_Drivers | Bookings | Cancelled | Revenue_Received | Revenue_Pending |
|---|---|---|---|---|---|
| 8 | 5 | 16 | 2 | 4522.00 | 1876.00 |

## Conclusion

Through these assignments I built the complete database of my Cab Booking System: I created the tables, protected them with constraints, filled them with realistic data, and wrote queries from simple filters to multi-table joins and subqueries. I learned that constraints stop wrong data before it enters the database, that GROUP BY with HAVING is useful for summaries, and that joins and subqueries are needed whenever a report needs data from more than one table. The queries also showed me why a relational database is a good choice for a booking and payment system, where accuracy and relationships matter.